UNIVERSIDAD NACIONAL DE INGENIERÍA
Facultad de Ingeniería Industrial y de Sistemas (FIIS)

TALLER 3: DESARROLLO E INTEGRACIÓN DE UNA API REST

Proyecto: Luxxito Sports Mobile & API REST

Sistema móvil y API REST en capas para la reserva de canchas deportivas, cotización horaria en servidor, control de stock de equipamiento por intervalo, pagos manuales verificables con Ticket QR y política de cancelación de 24 horas.

Curso: SI806 - Desarrollo Adaptativo e Integrado del Software
Stack: Flutter + Dart | Node.js + Express + TypeScript | Supabase PostgreSQL
Fecha: Noviembre 2026 - Lima, Perú
Equipo: Juan Medina, Elvis y Steven

1. FUNDAMENTACIÓN Y ANÁLISIS DEL PROBLEMA (LAS 8 PREGUNTAS)

1. ¿Qué problema concreto resolverán? La gestión manual de alquiler de canchas deportivas en Lima genera dobles reservas en el mismo horario, errores al cobrar tarifas diferenciadas (día vs. noche con iluminación), sobreventa de equipamiento adicional (chalecos, balones, árbitros), pérdida de turnos durante la revisión de comprobantes Yape/Plin y falta de trazabilidad en devoluciones por cancelación.

2. ¿Quién lo necesita y por qué? Los jugadores y organizadores de partidos necesitan consultar disponibilidad real desde su móvil, asegurar un bloqueo temporal de 20 minutos, agregar extras con stock verificado y recibir un Ticket QR de acceso. Los administradores requieren evitar cruces horarios y aplicar reglas automáticas de cobro y reembolso.

3. ¿Qué hará la API que no deba hacer el cliente? El servidor es la única autoridad sobre disponibilidad, precios (tarifa diurna < 18:00 y nocturna >= 18:00), cálculo de seña del 50%, control de stock de extras por intervalo, emisión idempotente del Ticket QR y cálculo de reembolso según la regla de las 24 horas. La app Flutter solo muestra datos y solicita operaciones.

4. ¿Qué datos y relaciones se almacenarán? Se almacenan 7 tablas en PostgreSQL (Supabase): usuarios, complejos (1:N canchas, 1:N extras), canchas (1:N reservas), extras (N:M reservas mediante reserva_extras) y pagos (N:1 reservas).

5. ¿Qué reglas ejecutará el servidor? (a) Cotización dinámica día/noche en servidor; (b) Exclusión mutua de horarios activos con bloqueo de 20 min; (c) Reserva de capacidad de extras por intervalo; (d) Protección en revisión y aprobación idempotente con QR; (e) Política de cancelación de 24 horas (100% reembolso >= 24h; 50% retención < 24h).

6. ¿Cómo evitar datos inconsistentes? Transacciones SQL (BEGIN/COMMIT/ROLLBACK) al crear reservas con extras, restricciones CHECK y claves foráneas en PostgreSQL, e idempotencia al aprobar pagos.

7. ¿Qué entradas deben rechazarse? DNI distinto de 8 dígitos, emails inválidos o tarifa nocturna menor a la diurna (400 Bad Request); horas pasadas o fuera del horario del complejo (422); turnos solapados o extras sin stock en el intervalo (409 Conflict).

8. ¿Cómo comprobarán cada resultado? Con 21 pruebas automatizadas en Vitest/Supertest (éxitos y fallos) ejecutadas en LOCAL_STORE. La colección Postman y el flujo en Android con base real requieren evidencia independiente; no se certifican por estas pruebas.

2. ARQUITECTURA EN CAPAS Y SEPARACIÓN DE RESPONSABILIDADES

• Cliente Móvil (Flutter + Dart en mobile/): Implementa las pantallas Explorar (filtros por distrito y deporte + creación CRUD de cancha), Disponibilidad y Reserva (selector de fecha, turnos, extras y cotización del servidor), Mis Reservas (pago Yape/Plin, Ticket QR, edición PUT, cancelación 24h y borrado DELETE) y Perfil.
• Capa de Controladores (backend/src/controllers/): Recibe solicitudes HTTP, formatea respuestas JSON y traduce excepciones de dominio a códigos HTTP estandarizados (200, 201, 400, 404, 409, 422, 500).
• Capa de Servicios (backend/src/services/): Contiene toda la lógica de negocio y validaciones en CatalogService y ReservationService.
• Capa de Acceso a Datos (backend/src/repositories/): Encapsula las consultas SQL y transacciones hacia PostgreSQL de Supabase.

3. MODELO DE DATOS RELACIONAL (SUPABASE POSTGRESQL)

1. **Tabla complejos:** id (PK), nombre, distrito, direccion, telefono, hora_apertura (7), hora_cierre (23), activo.
2. **Tabla canchas:** id (PK), complejo_id (FK -> complejos.id), nombre, deporte, superficie, techada, iluminacion, precio_hora_diurno (>0), precio_hora_nocturno (>= diurno), estado (ACTIVA/MANTENIMIENTO/INACTIVA).
3. **Tabla extras:** id (PK), complejo_id (FK -> complejos.id), nombre, descripcion, precio_unitario, stock_total (>=0), activo.
4. **Tabla reservas:** id (PK), cancha_id (FK -> canchas.id), cliente_nombre, cliente_dni (8 dígitos), cliente_telefono, cliente_email, fecha, hora_inicio, hora_fin, subtotal_cancha, subtotal_extras, total, monto_pagado, monto_devolucion, estado (PENDIENTE_PAGO, EN_REVISION, CONFIRMADA, CANCELADA, VENCIDA), codigo_qr, motivo_cancelacion, notas, expires_at.
5. **Tabla reserva_extras:** id (PK), reserva_id (FK -> reservas.id ON DELETE CASCADE), extra_id (FK -> extras.id), cantidad, precio_unitario, subtotal. Restricción UNIQUE(reserva_id, extra_id).
6. **Tabla pagos:** id (PK), reserva_id (FK -> reservas.id ON DELETE CASCADE), metodo (YAPE, PLIN, TRANSFERENCIA), modalidad (SENA_50, TOTAL), monto, numero_operacion, comprobante_url, estado (PENDIENTE_REVISION, APROBADO, RECHAZADO, REEMBOLSADO).

4. CATÁLOGO DE ENDPOINTS PROPIOS (OPERACIONES CRUD + NEGOCIO)

- **GET /api/health:** Estado del servidor y modo de conexión a Supabase (200)
- **GET /api/complejos:** Lista complejos deportivos activos, filtrable por distrito (200)
- **GET /api/canchas:** CRUD Read: Lista canchas con filtros de complejo, distrito y deporte (200)
- **POST /api/canchas:** CRUD Create: Registra nueva cancha validando tarifas diurna/nocturna (201, 400)
- **PUT /api/canchas/:id:** CRUD Update: Actualiza tarifas o estado de una cancha (200, 400, 404)
- **GET /api/canchas/:id/disponibilidad:** Negocio 1: Calcula turnos por fecha, tarifas día/noche y stock de extras (200, 404)
- **POST /api/reservas/cotizar:** Negocio 2: Cotiza cancha + extras en servidor y calcula seña del 50% (200, 409, 422)
- **GET /api/reservas:** CRUD Read: Lista reservas con sus extras y pagos asociados (200)
- **POST /api/reservas:** CRUD Create + Negocio 3: Crea reserva con bloqueo 20 min, sin solapamiento y validando stock (201, 400, 409, 422)
- **PUT /api/reservas/:id:** CRUD Update: Modifica teléfono de contacto o notas de la reserva (200, 404, 409)
- **DELETE /api/reservas/:id:** CRUD Delete: Elimina físicamente una reserva no confirmada (200, 404, 409)
- **POST /api/reservas/:id/pagos:** Negocio 4: Registra comprobante Yape/Plin/Transferencia y protege reserva en revisión (201, 409, 422)
- **PATCH /api/pagos/:id/aprobar:** Negocio 5: Aprobación idempotente de pago y generación de Ticket QR verificable (200, 404, 409)
- **POST /api/reservas/:id/cancelar:** Negocio 6: Cancela reserva aplicando regla de 24 horas y revoca el código QR (200, 404, 409)

5. DETALLE DE LAS REGLAS DE NEGOCIO IMPLEMENTADAS EN EL SERVIDOR

5.1. Cotización Autoritativa con Tarifa Diurna y Nocturna: El servidor recorre cada hora del intervalo [hora_inicio, hora_fin). Si la hora es menor a 18:00 aplica precio_hora_diurno; desde las 18:00 aplica precio_hora_nocturno. Además suma el subtotal de los extras solicitados y determina monto_sena_50 = round(total * 0.50, 2).

5.2. Exclusión Mutua de Turnos y Bloqueo Temporal de 20 Minutos: Antes de insertar una reserva, ReservationService verifica que no exista ninguna reserva en estado PENDIENTE_PAGO, EN_REVISION o CONFIRMADA en la misma cancha y fecha cuyo intervalo se cruce (hora_inicio < nueva_hora_fin AND hora_fin > nueva_hora_inicio). Si existe, responde 409 Conflict.

5.3. Control de Capacidad de Extras por Intervalo: Para evitar sobreventa de equipamiento compartido en un complejo (ej. árbitros o juegos de chalecos), el servidor suma las cantidades reservadas en todas las reservas activas solapadas de ese complejo en la misma fecha y hora. Si la cantidad solicitada supera el stock restante, responde 409 Conflict.

5.4. Conciliación Manual, Idempotencia y Ticket QR: Al enviar un comprobante (YAPE, PLIN o TRANSFERENCIA), se valida que el monto coincida con el 50% (SENA_50) o el 100% (TOTAL) y la reserva pasa a EN_REVISION. Al aprobar el pago, el método es idempotente: si ya estaba APROBADO, retorna el estado sin duplicar monto_pagado; si es la primera vez, acredita el monto, cambia la reserva a CONFIRMADA y genera el código QR (LUXXITO-QR-<ID>-<HASH>).

5.5. Política de Cancelación de 24 Horas: Al cancelar, el servidor calcula las horas restantes hasta el inicio del turno en zona horaria America/Lima (UTC-5). Si faltan >= 24 horas, calcula un reembolso del 100% del monto pagado (penalidad = 0). Si faltan < 24 horas, retiene el 50% por penalidad y reembolsa el 50%. En ambos casos revoca codigo_qr = null y libera el turno.

6. PRUEBAS AUTOMATIZADAS LOCALES (ÉXITOS Y FALLOS)

Caso 1 (Éxito - 200 OK): GET /api/complejos y GET /api/canchas?deporte=Futbol%207 retornan los complejos y canchas filtradas.

Caso 2 (Éxito CRUD - 201 y 200): POST /api/canchas crea una nueva cancha y PUT /api/canchas/:id actualiza sus tarifas diurna y nocturna.

Caso 3 (Fallo Validación - 400 Bad Request): POST /api/canchas rechaza una cancha cuando precio_hora_nocturno (90) es menor a precio_hora_diurno (120).

Caso 4 (Éxito Negocio - 201 Created): POST /api/reservas de 17:00 a 19:00 calcula 1h diurna (S/ 90) + 1h nocturna (S/ 120) + 2 chalecos (S/ 30) = S/ 240.00.

Caso 5 (Fallo Negocio - 409 Conflict): POST /api/reservas rechaza un segundo intento de reserva solapada de 20:00 a 21:00 sobre un turno ya reservado de 19:00 a 21:00.

Caso 6 (Fallo Negocio - 409 Conflict): POST /api/reservas rechaza la solicitud cuando dos canchas del mismo complejo piden más árbitros que el stock_total (2) en el mismo horario.

Caso 7 (Éxito Pago e Idempotencia - 201 y 200): POST /api/reservas/:id/pagos registra operación Yape y dos llamadas consecutivas a PATCH /api/pagos/:id/aprobar confirman la reserva con QR sin duplicar el saldo.

Caso 8 (Éxito Cancelación 24h - 200 OK): POST /api/reservas/:id/cancelar con >24h de anticipación aplica REEMBOLSO_100_MAYOR_24H, devuelve S/ 90.00 y revoca el QR.

7. DESPLIEGUE Y VALIDACIÓN DE INFRAESTRUCTURA - ISSUE #14

Responsable: Juan Medina (jc2709). Rama juan/despliegue-apk-validacion y PR #17 a develop. Revisión: 3 de noviembre de 2026.

API pública: https://luxxito-sports-api.onrender.com/api. GET /health comprobado con HTTP 200 y db_mode POSTGRES_POOL. Primer despliegue Render del commit f4a507a de main.

Servicio: Render Free, Node 22, raíz backend, build npm ci --include=dev && npm run build, inicio node dist/server.js y health /api/health. Las variables privadas solo se configuran en Render; no están en Git.

Workflow del APK: Los secretos pasan por env y los argumentos --dart-define se entrecomillan. Una validación previa rechaza URLs con espacios/paréntesis o IPs locales. API_BASE_URL apunta a la API pública.

Consumo de API propia: El APK no cambia silenciosamente a Supabase directo. El teléfono usa HTTPS y dispone de 60 segundos de timeout para el despertar del servicio gratuito. La descarga debe provenir del release latest-apk actualizado.

Historial integrado: PR #11: promoción de identidad, QR y expiración; PR #12: endpoint comprobantes/RPC; PR #13: promoción; PR #15/#16: enfoque solo Android. La integración de un endpoint no demuestra por sí sola que su función esté completa.

Comprobaciones de este cambio: Build backend correcto; 21/21 pruebas API locales y 3/3 pruebas de configuración. CI del PR verde para el commit 2a2b341. Resultado final del APK y promoción: consultar el informe de validación actualizado.

Pendiente Android físico: Instalar el APK actual, registrar modelo/versión y capturar el recorrido. No se declara completada la prueba de punta a punta sin esa evidencia.

Fuera de este cambio: Los fallos de pagos, upload de comprobantes y persistencia QR se atenderán después, por indicación de Juan. No se presentan aquí como corregidos; el criterio completo de aceptación de la issue sigue abierto.

Evidencia: docs/validaciones/2026-11-03-infra-issue14.md y capturas/infra-render-live.jpg. El PDF distingue pruebas locales, servicio publicado y validación física pendiente.